Ex. No. 9	Accessing APIs through HTTP
Date of Exercise	12/10/26
GitHub Host Link	https://github.com/jomiwilson/flutter_lab_exp9

Aim

To access a REST API using HTTP and display JSON data in Flutter.

Description

This experiment introduces accessing a REST API in Flutter using the http package. An HTTP GET request is sent to the API and the JSON response is decoded using jsonDecode. The data is displayed using ListView.builder, with loading and error messages handled in the UI.

Tools/Software Required: Flutter

Procedure:

Step 1

Create:

```
flutter create api_app
cd api_app
```

Step 2 - Add HTTP package

In pubspec.yaml:

```
dependencies:
  http:
```

Run:

```
flutter pub get
```

Step 3 - Import packages

```
import 'dart:convert';
import 'package:http/http.dart' as http;
```

Step 4 - Create API function

```
Future<void> fetchData() async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/posts'),
  );
  if (response.statusCode == 200) {
    final data = jsonDecode(response.body);
    print(data);
  } else {
    print('Failed to load data');
  }
```

```
}
```

Step 5 - Call the function

Call:

```
fetchData();
```

Step 6 - Display API data

Use:

```
ListView.builder(
  itemCount: data.length,
  itemBuilder: (context, index) {
    return ListTile(
      title: Text(data[index]['title']),
    );
  },
)
```

Step 7 - Handle loading

Display:

```
Loading...
```

while the API request is being processed.

Step 8 - Handle errors

Display an error message when the HTTP status is not successful.

Program

pubspec.yaml (dependencies)

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.2
```

lib/main.dart

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
```

```
      home: ApiPage(),
    );
  }
}

class ApiPage extends StatefulWidget {
  const ApiPage({super.key});

  @override
  State<ApiPage> createState() => _ApiPageState();
}

class _ApiPageState extends State<ApiPage> {
  List<dynamic> data = [];
  bool isLoading = true;
  String errorMessage = '';

  @override
  void initState() {
    super.initState();
    fetchData();
  }

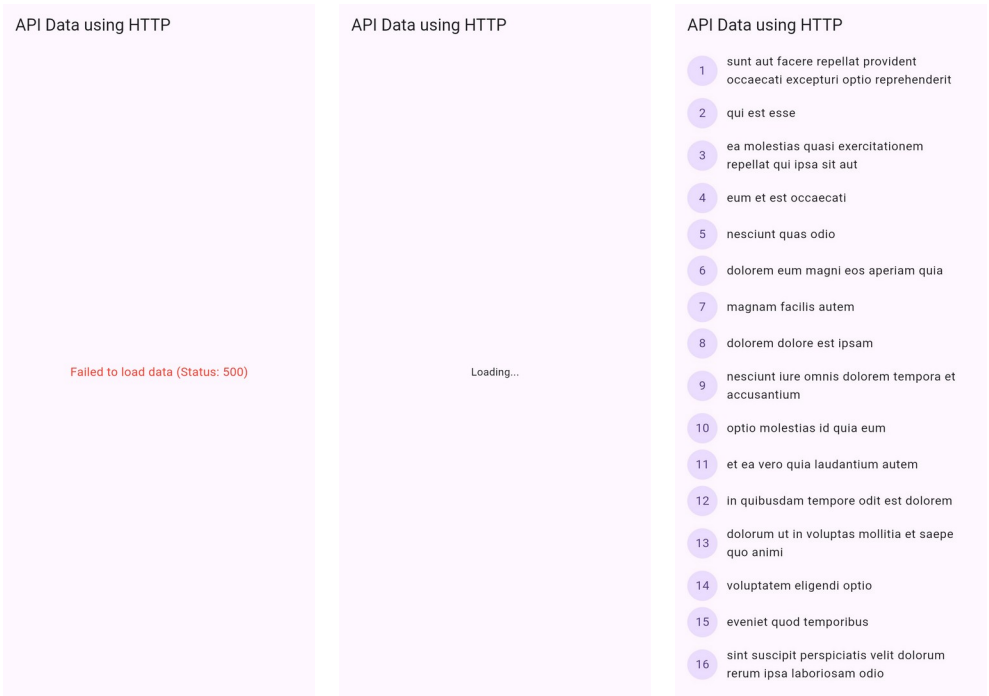
  // Send an HTTP GET request and decode the JSON response
  Future<void> fetchData() async {
    try {
      final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/posts'),
      );
      if (response.statusCode == 200) {
        setState(() {
          data = jsonDecode(response.body);
          isLoading = false;
        });
      } else {
        setState(() {
          errorMessage = 'Failed to load data (Status: $
{response.statusCode})';
          isLoading = false;
        });
      }
    } catch (e) {
      setState(() {
        errorMessage = 'Failed to load data';
        isLoading = false;
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    Widget body;
    if (isLoading) {
```

```
      body = const Center(child: Text('Loading...'));
    } else if (errorMessage.isNotEmpty) {
      body = Center(
        child: Text(errorMessage, style: const TextStyle(color: Colors.red)),
      );
    } else {
      body = ListView.builder(
        itemCount: data.length,
        itemBuilder: (context, index) {
          return ListTile(
            leading: CircleAvatar(child: Text('${data[index]['id']}')),
            title: Text(data[index]['title']),
          );
        },
      );
    }

    return Scaffold(
      appBar: AppBar(
        title: const Text('API Data using HTTP'),
      ),
      body: body,
    );
  }
}
```

Output



Result

A Flutter application for accessing a REST API using HTTP was developed successfully. The application sent an HTTP GET request and decoded the JSON response using jsonDecode. The retrieved post titles were displayed in the UI using ListView.builder. A loading message was shown during the request and an error message was shown on failure.